

Single- and Multiple-Processor Models for Minimizing Completion Time Variance

Nicholas G. Hall

*Academic Faculty of Management Sciences, The Ohio State University,
Columbus, Ohio 43210*

This article concerns the scheduling of n jobs around a common due date, so as to minimize the average total earliness plus total lateness of the jobs. Optimality conditions for the problem are developed, based on its equivalence to an easy scheduling problem. It seems that this problem inherently has a huge number of optimal solutions and an algorithm is developed to find many of them. The model is extended to allow for the availability of multiple parallel processors and an efficient algorithm is developed for that problem. In this more general case also, the algorithm permits great flexibility in finding an optimal schedule.

1. INTRODUCTION AND MOTIVATION

Consider a single machine scheduling problem involving n jobs with known processing times p_j , $j = 1, \dots, n$, and a common due date d for those jobs. We assume that $d \geq \sum_{j=1}^n p_j$, and that preemption is not allowed. We also assume that no ready-date restrictions apply. A problem considered by Kanet [2] is the finding of a schedule S to minimize an average penalty given by

$$Z(S) = \sum_{j=1}^n \text{abs} \frac{(C_j - d)}{n}, \quad (1)$$

where C_j represents the completion time of job j , and abs denotes the absolute value function.

This formulation has applications in industrial and logistical situations involving scheduling, where the completion of a job either before or after its due date is costly. It is widely recognized that completion after a due date is likely to incur costs in the loss of the order and of customer goodwill. On the other hand, completion before the due date may lead to higher inventory costs and, if goods are perishable, potential loss of usable production. Early completion of jobs also increases the danger of overstocking in the event of order cancellation.

2. ALGORITHM

The algorithm devised by Kanet for Eq. (1) requires several definitions. The set B represents an ordered set of jobs to be scheduled consecutively so that the last job in B completes processing at time d . The set A similarly represents an ordered set of jobs to be scheduled consecutively so that the first job in A starts processing at time d . The

set U represents the hitherto unscheduled jobs. Kanet's algorithm is

```

begin
   $B \leftarrow A \leftarrow \phi$ 
  while ( $U \neq \phi$ ) do
    remove a job  $k$  from  $U$  such that  $p_k = \max_j \{p_j\}$ 
    insert job  $k$  into the last position in  $B_j$ 
    if ( $U \neq \phi$ ) do
      remove a job  $k$  from  $U$  such that  $p_k = \max_j \{p_j\}$ 
      insert job  $k$  into the first position in  $A$ 
    end
  end
   $S \leftarrow (B, A)$ 
end

```

Informally, the algorithm selects jobs by nonincreasing order of processing time. Then the first job is scheduled to end at time d , the second to begin at time d , the third to end at time d (pushing the first job earlier), the fourth to begin at time d (pushing the second job later), and so on until all jobs have been scheduled.

3. PROOF OF OPTIMALITY

Kanet [2] demonstrates two conditions satisfied by at least one optimal solution to Eq. (1):

(a) A consecutive scheduling of all jobs: If there is a gap in the schedule, penalty savings will result from closing it.

(b) A job will complete processing at time d : If this is not the case, the entire schedule can be moved earlier or later, and in at least one direction average penalty will not increase.

Kanet's proof of optimality for the algorithm continues by defining three mathematical properties imposed on the solution by the algorithm. These three properties are then claimed to be both necessary and sufficient for optimality. The properties are:

- 1A Jobs in B are ordered by nonincreasing processing time (LPT).
- 1B Jobs in A are ordered by nondecreasing processing time (SPT).
- 2A If n is even, $|B| = |A|$, where $|R|$ denotes the cardinality of the set R .
- 2B If n is odd, $|B| = |A| + 1$.
- 3 There is a one-to-one mapping of the jobs in A into the jobs in B such that $k \in A \rightarrow j \in B \Rightarrow p_k \leq p_j$.

4. AN EQUIVALENT PROBLEM

Given conditions (a) and (b), the problem of minimizing Eq. (1) is equivalent to defining the position of each job in the consecutive sequence and specifying which job completes processing at time d . This can be considered as the minimization of

$$Z'(S) = \frac{1}{n} \sum_{j=1}^n p_j X_j \quad (2)$$

where the X_j 's are penalties dependent on the position of job j . Specifically, if job j is completed before time d , we let X_j represent the number of jobs scheduled to complete processing before job j (including job j itself). Similarly, if job j is completed after time d , we let X_j represent the number of jobs scheduled to complete processing after job j (including job j itself). The first job in a schedule will have no penalty, i.e., $X_j = 0$. Any schedule will also have two jobs with $X_j = 1$ (the second and last), two jobs with $X_j = 2$ (the third and second last), and so on. It should be clear that the set of available penalties for a particular problem is dependent on the number of jobs to be scheduled, but not on their processing times. The complete sequence of available penalties for n jobs is

$$0, 1, 1, 2, 2, \dots, \left\lceil \frac{n-1}{2} \right\rceil, \quad (3)$$

where $\lceil x \rceil$ is the smallest integer greater than or equal to x . Using this interpretation of the problem, we can define an optimal policy by means of a lemma.

LEMMA 1: Arrange the penalties in nondecreasing order, as in Eq. (3). Then Eq. (2), and equivalently Eq. (1), are minimized by a schedule arranging the jobs in nonincreasing order of processing time (LPT) against those penalties.

PROOF: Assume the jobs are not LPT ordered, then there exists a pair of jobs in positions $i, i+1$ such that $p_i < p_{i+1}$. Interchanging the jobs causes a change in average penalty (2) of

$$\frac{1}{n}(X_i p_{i+1} - X_i p_i + X_{i+1} p_i - X_{i+1} p_{i+1}) = \frac{1}{n}(X_i - X_{i+1})(p_{i+1} - p_i) \leq 0.$$

Therefore an LPT ordering of the jobs against the penalties is at least as good as any other ordering. The usefulness of ordering by LPT (and the opposite rule, SPT) is discussed by Baker [1]. The scheduling of two sequences to minimize the sum of the products of individual terms has been used by Panwalkar et al. [3].

It seems that the optimality properties 1A, 1B, 2A, 2B, and 3 of Kanet are not necessary. In particular, if the number of jobs n is even, property 2A need not hold because the shortest job may be placed either to complete processing at time d or to start processing at time d , with the same penalty being incurred. For example, two jobs J1, J2 with processing times 12 and 10, respectively, will have average penalty of 5 if $B = \{J1, J2\}$, $A = \{\phi\}$ or if $B = \{J1\}$, $A = \{J2\}$. Further, the properties do not seem to be sufficient for optimality. Consider the data set from [2]:

Job	J1	J2	J3	J4	J5
Processing Time	7	12	5	4	10
Due date =	39				

The schedule in Figure 1 establishes properties 1A, 1B, 2A, 2B, and 3:

J2	J5	J1	J4	J3	
10	22	32	39	43	48

Figure 1. Schedule satisfying the optimality properties of [2] with average penalty of $\frac{37}{5}$.

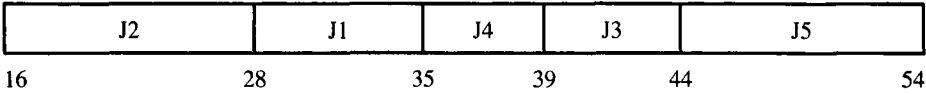


Figure 2. Optimal schedule with average penalty of $\frac{35}{5} = 7$.

The algorithm of [2], however, gives the optimal schedule in Figure 2.

5. ALTERNATIVE OPTIMAL SOLUTIONS

The fact that there are many pairs of numbers in the sequence of available penalties [Eq. (3)] means that there are numerous alternative ways to schedule the jobs and still achieve the ordering specified by Lemma 1. We will schedule the jobs so that when the number of jobs scheduled so far is odd, the next job can be scheduled *either* to complete processing at time *d* or to start processing at time *d*, and the algorithm will still deliver an optimal schedule. This flexibility may enable a scheduler to take into account constraints or conditions not included in the model at present, such as ready dates for jobs or priority or precedence constraints. We summarize the new algorithm:

```
begin
  B ← A ← φ
  while (U ≠ φ) do
    remove a job k from U such that  $p_k = \max_j \{p_j\}$ 
    if  $|B| = |A|$  insert job k into the last position in B
    if  $|B| = |A| + 2$  insert job k into the first position in A
    else (arbitrary choice) insert job k into the last position in B or the first position
      in A
    end
  end
  S ← (B,A)
end
```

The new algorithm provides three alternative solutions with the same value as the original optimal solution, as we demonstrate in Figure 3.

The average penalty for each of these solutions is $\frac{35}{5} = 7$, as for the original optimal solution (Figure 2). If we denote by $\lfloor x \rfloor$ the greatest integer less than or equal to *x*,

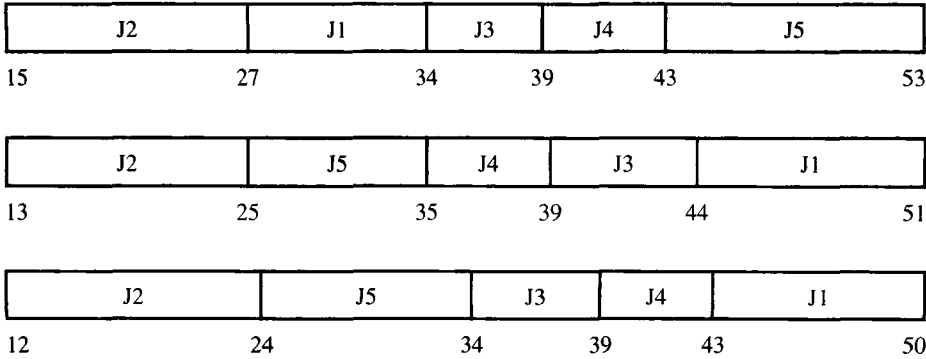


Figure 3. Alternative optimal solutions delivered by the new algorithm.

we can see that the new algorithm will provide up to $2^{\lfloor n/2 \rfloor}$ alternative optimal solutions in place of the original one. This is true because at every even-numbered step (e.g., the second largest or fourth largest job) in the algorithm two alternative solutions are available from every partial solution at the previous step.

6. SCHEDULING WITH MULTIPLE PROCESSORS

We consider the minimization of the average penalty [Eq. (1)] when identical processors denoted by the set $M = \{1, \dots, m\}$ are available. We assume that the processors can all be used in parallel. As before, all processing times are known, no ready date restrictions apply, and preemption is not allowed. We let $B_i, i = 1, \dots, m$ denote an ordered set of jobs scheduled consecutively on machine i so that the last job in B_i is completed at time d . Similarly, let $A_i, i = 1, \dots, m$ denote an ordered set of jobs scheduled consecutively on machine i so that the first job in A_i starts at time d .

As in the single-processor case there will be at least one optimal schedule with consecutive jobs and with a job completing processing at time d ; this will be true for all processors. The problem is equivalent to Eq. (2) except that this time the available penalties are

$$0, \dots, 0, 1, \dots, 1, 2, \dots, 2, \dots, \left\lceil \frac{n-m}{2m} \right\rceil, \quad (4)$$

That is, each penalty is available m times as often as it was in the single-processor case. Penalty 0 appears m times, penalty 1 appears $2m$ times, penalty 2 appears $2m$ times, and so on. However, the minimization problem is just the same as before, except that even more arbitrary choices are now available. The following algorithm will solve the problem with n jobs and m processors:

```

begin
   $B_i \leftarrow \phi, \quad i = 1, \dots, m$ 
   $A_i \leftarrow \phi, \quad i = 1, \dots, m$ 
  while ( $U \neq \phi$ ) do
    remove a job  $k$  from  $U$  such that  $p_k = \max_{j \in U} \{p_j\}$ 
     $Q \leftarrow \{i | i \in M, |B_i| = \min_{i \in M} \{|B_i|\}\}$ 
     $R \leftarrow \{i | i \in M, |A_i| = \min_{i \in M} \{|A_i|\}\}$ 
    if  $|B_i|, i \in Q = |A_i|, i \in R$ , insert job  $k$  into the last position in (arbitrary choice)
       $B_i, i \in Q$ 
    if  $|B_i|, i \in Q = |A_i| + 2, i \in R$ , insert job  $k$  into the first position in (arbitrary
      choice)  $A_i, i \in R$ 
    else (arbitrary choice), insert job  $k$  into the last position in  $B_i$  for any  $i \in Q$  or
      the first position in  $A_i$  for any  $i \in R$ 
    end
  end
   $S \leftarrow (B_1, A_1), \dots, (B_m, A_m)$ 
end

```

7. NUMERICAL EXAMPLE

If two processors are available ($m = 2$), the example problem of Kanet will give many alternative optimal solutions. We assume for this example that all arbitrary choices for assignment to processors are made in favor of the lower-indexed machine and that

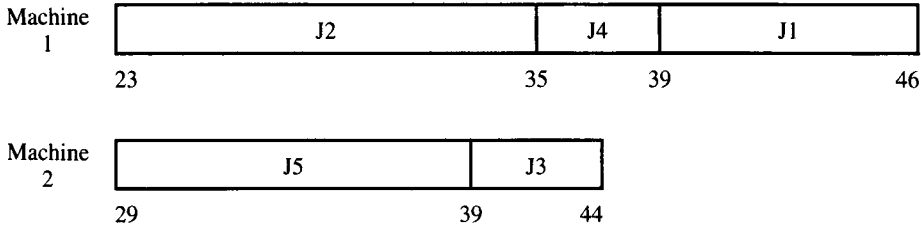


Figure 4. Optimal solution for the example problem with two processors.

the arbitrary choice under the else statement in the algorithm is to A_r , $r \in R$. The solution generated is illustrated in Figure 4.

The solution has average penalty of $\frac{16}{5}$. As we would expect, the average penalty is reduced by the availability of another processor. While $m < n$, the availability of another processor will always reduce average penalty.

8. SUMMARY

The problem of minimizing average earliness plus lateness penalty around a given due date is examined. This problem is equivalent to ordering by LPT sequence against a fixed schedule of known penalties. An algorithm is developed that provides the scheduler with many ways to achieve an optimal solution, widening the applicability of the model. When several identical processors are available, the same scheduling rule continues to provide a variety of alternative optimal solutions.

REFERENCES

[1] Baker, K. R., *Introduction to Sequencing and Scheduling*, Wiley, New York, 1974.
[2] Kanet, J. J., "Minimizing the average deviation of job completion times about a common due date," *Naval Research Logistics Quarterly*, **28**(4), 643-651 (1981).
[3] Panwalkar, S. S., Smith, M. L., and Seidmann, A., "Common due date assignment to minimize total penalty for the one machine scheduling problem," *Operations Research*, **30**(2), 391-399 (1982).